

47. 在一个充满活力的新软件公司, 一个新女雇员的初始工资为 50 000 美元, 公司允诺每年底她的工资将是她前一年工资的 2 倍, 并且她在公司的每年都将额外增加 10 000 美元。

a) 构造一个与被雇用的第 n 年她的工资数有关的递推关系。

b) 求解这个递推关系, 找出她被雇用的第 n 年的工资。

某些线性递推关系没有常数系数, 但也可以系统地求解。这就是形如 $f(n)a_n = g(n)a_{n-1} + h(n)$ 的递推关系的情况。练习 48~50 说明了这一点。

* 48. a) 证明递推关系

$$f(n)a_n = g(n)a_{n-1} + h(n)$$

其中 $n \geq 1$, $a_0 = C$, 可以转变成如下形式的递推关系

$$b_n = b_{n-1} + Q(n)h(n)$$

其中 $b_n = g(n+1)Q(n+1)a_n$, 满足

$$Q(n) = (f(1)f(2)\cdots f(n-1)) / (g(1)g(2)\cdots g(n))$$

b) 使用 a 求解原来的递推关系以便得到

$$a_n = \frac{C + \sum_{i=1}^n Q(i)h(i)}{g(n+1)Q(n+1)}$$

* 49. 使用练习 48 求解递推关系 $(n+1)a_n = (n+3)a_{n-1} + n$, $n \geq 1$, $a_0 = 1$ 。

50. 可以证明当以随机顺序排序 n 个元素时, 快速排序算法(在 5.4 节练习 50 中描述)所做的平均比较次数满足递推关系

$$C_n = n + 1 + \frac{2}{n} \sum_{k=0}^{n-1} C_k$$

$n=1, 2, \dots$, 且初始条件 $C_0=0$ 。

a) 证明 $\{C_n\}$ 也满足递推关系 $nC_n = (n+1)C_{n-1} + 2n$, $n=1, 2, \dots$ 。

b) 使用练习 48 求解 a 的递推关系以找到关于 C_n 的显示公式。

** 51. 证明定理 4。

52. 证明定理 6。

53. 求解具有初始条件 $T(1)=6$ 的递推关系 $T(n)=nT^2(n/2)$ 。[提示: 令 $n=2^k$, 然后做替换 $a_k = \log T(2^k)$ 以便得到一个线性非齐次的递推关系。]

8.3 分治算法和递推关系

8.3.1 引言

许多递归算法把一个给定输入的问题划分成一个或多个小问题。连续使用这种划分直到可以很快地找到这些较小问题的解。例如, 在执行一个二分检索时把对一个元素在表中的搜索归约为对该元素在长度减半的表中的搜索。我们继续使用这种分解直到只剩下一个元素。当我们使用归并排序算法排序一个整数的表时, 我们将这个表划分成相等大小的两半并且分别排序每一半。然后将两个排好序的半个表归并。这种类型的递归算法的另一个例子就是整数乘法的过程, 它将两个整数相乘的问题分解成三组位数减半的整数相乘。这种分解连续使用直到只剩下一个整数为止。这些过程叫作分治算法, 因为它们将一个问题划分成较小规模的同一问题的一个或多个实例, 然后用这些小问题的解来处理这个问题以找到初始问题的解, 这当中也许需要一些额外的工作。

这一节将说明怎样用递推关系分析分治算法的计算复杂度。我们将用这些递推关系估计许多不同的分治算法(包括我们在本节引入的算法)所使用的运算次数。

8.3.2 分治递推关系

假设一个递归算法把一个规模为 n 的问题分成 a 个子问题, 其中每个子问题的规模是 n/b (为简单起见, 假设 n 是 b 的倍数。实际上, 较小问题的规模常常是小于等于或者大于等于 n/b 的最近的整数)。此外, 假设在把子问题的解组合成原来问题的解的算法处理步中需要总量为

$g(n)$ 的额外的运算。那么,如果 $f(n)$ 表示求解规模为 n 的问题所需的运算数,则得出 f 满足递推关系

$$f(n) = af(n/b) + g(n)$$

这就叫作分治递推关系。

首先我们将建立一些可用于研究某些重要算法复杂度的分治递推关系。然后将说明怎样用这些分治递推关系估计这些算法的复杂度。

Extra
Examples

例 1 二分搜索 在 3.1 节我们引入了二分搜索算法。当 n 是偶数时,二分搜索算法把对某个元素在长度为 n 的搜索序列中的搜索转变成对同一元素在长度为 $n/2$ 的搜索序列中的二分搜索。(因此,规模为 n 的问题已经被分解成规模为 $n/2$ 的问题。)执行这个分解需要 2 次比较(一次是为了确定要用到表的哪一半,另一次是为了确定表中是否还有项留下来)。所以,如果 $f(n)$ 是在规模为 n 的搜索序列中搜索一个元素所需要的比较次数,那么当 n 是偶数时 $f(n) = f(n/2) + 2$ 。

例 2 找一个序列的最大和最小 考虑下面的查找序列 $a_1, a_2, \dots, a_n$ 中最小和最大元素的算法。如果 $n=1$,那么 a_1 就是最大和最小的元素。如果 $n>1$,把这个序列分成两个序列,或者两者有同样多的元素,或者一个序列比另一个序列多一个元素。问题就归约成查找两个较小序列的最大和最小元素。比较两个较小集合的最大和最小元素,从而得到全体的最大和最小元素,原问题的解就得到了。

设 $f(n)$ 是找 n 元素序列的最小和最大元素所需要的总的比较次数。我们已经说明了当 n 是偶数时一个规模为 n 的问题可以归约成两个规模为 $n/2$ 的问题,这里要使用 2 次比较,一次是比较两个序列的最小元素,而另一次是比较两个序列的最大元素。当 n 是偶数时就得到递推关系 $f(n) = 2f(n/2) + 2$ 。

例 3 归并排序 在 5.4 节介绍的归并排序算法把一个具有 n 个项(其中 n 为偶数)的待排序的表划分成两个表,每个表 $n/2$ 个元素,并且用少于 n 次的比较将两个排好序的表归并成一个排好序的表。因此,通过归并排序算法排序 n 个元素的表用到的比较次数小于 $M(n)$,其中函数 $M(n)$ 满足分治递推关系

$$M(n) = 2M(n/2) + n$$

例 4 整数的快速乘法 令人惊讶的是,存在许多比传统的整数乘法算法(在 4.2 节描述过)更有效的算法。这里描述的一个有效的算法就用到了分治技术。这个快速的乘法算法把每个 $2n$ 位的二进制整数分成两块,每块 n 位。然后,原来 $2n$ 位的二进制整数的乘法被分解成 3 个 n 位二进制数的乘法,还要加上移位和加法。

假设 a 和 b 是两个整数的 $2n$ 位的二进制表达式(为了使它们等长,如果需要,可以在这些表达式前面加上若干个 0)。令

$$a = (a_{2n-1} a_{2n-2} \cdots a_1 a_0)_2, \quad b = (b_{2n-1} b_{2n-2} \cdots b_1 b_0)_2$$

令

$$a = 2^n A_1 + A_0, \quad b = 2^n B_1 + B_0$$

其中

$$\begin{aligned} A_1 &= (a_{2n-1} \cdots a_{n+1} a_n)_2, & A_0 &= (a_{n-1} \cdots a_1 a_0)_2 \\ B_1 &= (b_{2n-1} \cdots b_{n+1} b_n)_2, & B_0 &= (b_{n-1} \cdots b_1 b_0)_2 \end{aligned}$$

快速整数乘法算法是基于恒等式

$$ab = (2^{2n} + 2^n)A_1 B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0 B_0$$

关于这个恒等式的一个重要的事实就是,它证明了两个 $2n$ 位整数的乘法可以用 3 个 n 位整数的乘法加上加法、减法以及移位来实现。这证明了如果 $f(n)$ 是两个 n 位整数相乘所需要的按位运算的总数,那么



$$f(2n) = 3f(n) + C_n$$

这个等式成立的理由是：3次 n 位整数的乘法可以使用 $3f(n)$ 次按位运算实现。每次加法、减法和移位使用的运算次数是 n 位运算的常数倍，而 C_n 表示由这些运算用到的总的按位运算数。

例5 快速矩阵乘法 在3.3节例7中，我们证明了使用矩阵乘法的定义进行两个 $n \times n$ 矩阵相乘需要 n^3 次乘法和 $n^2(n-1)$ 次加法。因此，按照这种方法计算两个 $n \times n$ 矩阵之积需要 $O(n^3)$ 次运算(乘法和加法)。令人惊讶的是，对于两个 $n \times n$ 矩阵相乘存在更有效的分治算法。这个由沃尔克·斯特拉森于1969年提出的算法当 n 是偶数时将两个 $n \times n$ 矩阵的相乘归约为两个 $(n/2) \times (n/2)$ 矩阵的7次相乘和 $(n/2) \times (n/2)$ 矩阵的15次相加。(要了解这个算法的细节见[CoLeRiSt09]。)于是，如果 $f(n)$ 是用到的运算(乘法和加法)次数，那么当 n 是偶数时有

$$f(n) = 7f(n/2) + 15n^2/4$$

如例1~5所示，在许多不同的情况中都出现了形如 $f(n) = af(n/b) + g(n)$ 的递推关系。可以对满足这种递推关系的函数的阶做出估计。假设 f 满足这个递推关系，其中 n 可被 b 整除。令 $n = b^k$ ，其中 k 是一个正整数。那么

$$\begin{aligned} f(n) &= af(n/b) + g(n) \\ &= a^2 f(n/b^2) + ag(n/b) + g(n) \\ &= a^3 f(n/b^3) + a^2 g(n/b^2) + ag(n/b) + g(n) \\ &\vdots \\ &= a^k f(n/b^k) + \sum_{j=0}^{k-1} a^j g(n/b^j) \end{aligned}$$

由于 $n/b^k = 1$ ，所以

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j g(n/b^j)$$

我们可以使用这个关于 $f(n)$ 的等式估计满足分治关系的函数的阶。

定理1 设 f 是满足递推关系

$$f(n) = af(n/b) + c$$

的增函数，其中 n 被 b 整除， $a \geq 1$ ， b 是大于1的整数， c 是一个正实数。那么

$$f(n) \text{ 是 } \begin{cases} O(n^{\log_b a}) & a > 1 \\ O(\log n) & a = 1 \end{cases}$$

而且，当 $n = b^k$ (其中 k 是正整数)， $a \neq 1$ 时

$$f(n) = C_1 n^{\log_b a} + C_2$$

其中 $C_1 = f(1) + c/(a-1)$ 且 $C_2 = -c/(a-1)$ 。

证明 首先令 $n = b^k$ 。由定理前面的讨论中得到的关于 $f(n)$ 的表达式和 $g(n) = c$ ，我们有

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j c = a^k f(1) + c \sum_{j=0}^{k-1} a^j$$

当 $a=1$ 时，有

$$f(n) = f(1) + ck$$

由于 $n = b^k$ ，所以有 $k = \log_b n$ 。于是

$$f(n) = f(1) + c \log_b n$$

当 n 不是 b 的幂时，对某个正整数 k 有 $b^k < n < b^{k+1}$ 。由于 f 是递增的，所以 $f(n) \leq f(b^{k+1}) = f(1) + c(k+1) = (f(1) + c) + ck \leq (f(1) + c) + c \log_b n$ 。因此，在两种情况下当 $a=1$ 时 $f(n)$ 都是 $O(\log n)$ 。

现在假设 $a > 1$ 。首先假定 $n = b^k$ ， k 是正整数。由几何级数的求和公式(2.4节定理1)得到

$$\begin{aligned}
 f(n) &= a^k f(1) + c(a^k - 1)/(a - 1) \\
 &= a^k [f(1) + c/(a - 1)] - c/(a - 1) \\
 &= C_1 n^{\log_a a} + C_2
 \end{aligned}$$

因为 $a^k = a^{\log_a n} = n^{\log_a a}$ (见附录 B 的练习 4), 其中 $C_1 = f(1) + c/(a - 1)$ 且 $C_2 = -c/(a - 1)$ 。

现在假设 n 不是 b 的幂。那么 $b^k < n < b^{k+1}$, 其中 k 是一个非负整数。由于 f 是递增的, 所以

$$\begin{aligned}
 f(n) &\leq f(b^{k+1}) = C_1 a^{k+1} + C_2 \\
 &\leq (C_1 a) a^{\log_a n} + C_2 \\
 &\leq (C_1 a) n^{\log_a a} + C_2
 \end{aligned}$$

因为 $k \leq \log_b n < k+1$ 。

于是, $f(n)$ 是 $O(n^{\log_a a})$ 。

例 6~9 说明怎样使用定理 1。

例 6 设 $f(n) = 5f(n/2) + 3$ 且 $f(1) = 7$, 求 $f(2^k)$, 其中 k 是一个正整数。如果 f 是一个增函数, 请估计 $f(n)$ 。

解 根据定理 1 的证明, 当 $a=5, b=2, c=3$ 时, 我们看到如果 $n=2^k$, 那么

$$\begin{aligned}
 f(n) &= a^k [f(1) + c/(a - 1)] + [-c/(a - 1)] \\
 &= 5^k [7 + (3/4)] - 3/4 \\
 &= 5^k (31/4) - 3/4
 \end{aligned}$$

而且, 如果 $f(n)$ 是递增的, 那么定理 1 证明 $f(n)$ 是 $O(n^{\log_2 5}) = O(n^{\log 5})$ 。

我们可以使用定理 1 估计二分搜索算法和例 2 查找序列的最小和最大元素的算法的计算复杂度。

例 7 估计二分搜索使用的比较次数。

解 在例 1 中证明了当 n 是偶数时 $f(n) = f(n/2) + 2$, 其中 $f(n)$ 是在规模为 n 的序列中实现二分搜索需要的比较次数。因此得出 $f(n)$ 是 $O(\log n)$ 。

例 8 估计用例 2 给定的算法查找序列的最大和最小元素所使用的比较次数。

解 在例 2 中我们证明了当 n 是偶数时 $f(n) = 2f(n/2) + 2$, 其中 f 是算法需要的比较次数。于是, 由定理 1 得到 $f(n)$ 是 $O(n^{\log 2}) = O(n)$ 。

我们现在叙述一个更一般的、更复杂的定理, 定理 1 是它的特例。这个定理(或者更强的版本, 包括大 Θ 估计)有时称为主定理(master theorem), 因为它在分析许多重要的分治算法的复杂度中很有用。

定理 2 主定理 设 f 是满足递推关系

$$f(n) = af(n/b) + cn^d$$

的增函数, 其中 $n = b^k$, k 是一个正整数, $a \geq 1$, b 是大于 1 的整数, c 和 d 是实数, 满足 c 是正的且 b 是非负的。那么

$$f(n) \text{ 是 } \begin{cases} O(n^d) & a < b^d \\ O(n^d \log n) & a = b^d \\ O(n^{\log_b a}) & a > b^d \end{cases}$$

定理 2 的证明留给读者作为练习 29~33。

例 9 归并排序的复杂度 在例 3 中我们解释了用归并排序来对 n 个元素的表进行排序所使用的比较次数少于 $M(n)$, 其中 $M(n) = 2M(n/2) + n$ 。根据主定理(定理 2), 我们发现 $M(n)$ 是 $O(n \log n)$, 这与在 5.4 节得到的估计一致。

例 10 估计使用例 4 描述的快速乘法算法进行两个 n 位整数相乘所需要的按位运算的次数。

解 例 4 证明了当 n 是偶数时 $f(n) = 3f(n/2) + Cn$, 其中 $f(n)$ 是使用快速乘法算法进行两

个 n 位整数相乘所需的按位运算的次数。于是, 由定理 2 得到 $f(n)$ 是 $O(n^{\log 3})$ 。注意 $\log 3 \sim 1.6$ 。因为传统的乘法算法使用 $O(n^2)$ 次按位运算, 所以对于足够大的整数, 包括实际应用中出现的大整数, 快速乘法算法在时间复杂度方面比传统的算法有了本质的改进。◀

例 11 估计使用例 5 的矩阵乘法算法进行两个 $n \times n$ 矩阵相乘所需要的乘法和加法的次数。

解 令 $f(n)$ 表示使用例 5 提到的算法进行两个 $n \times n$ 矩阵相乘所需的加法和乘法的次数。当 n 是偶数时, 我们有 $f(n) = 7f(n/2) + 15n^2/4$ 。于是由定理 2 得到 $f(n)$ 是 $O(n^{\log 7})$ 。注意 $\log 7 \sim 2.8$ 。由于传统的两个 $n \times n$ 矩阵相乘的算法要用 $O(n^3)$ 次加法和乘法, 显然, 对足够大的整数 n , 包括出现在许多实际应用中的大整数, 这个算法比传统的算法在时间复杂度方面更加有效。◀

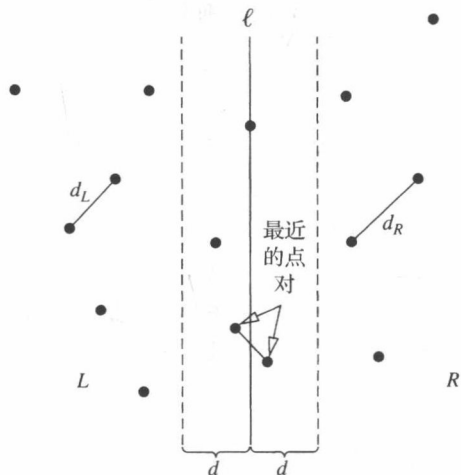
最近点对问题 我们在结束这一节之前引入一个来自计算几何的分治算法, 计算几何是离散数学的一部分, 是专注于求解几何问题的算法。

例 12 最近点对问题 考虑确定平面上 n 个点 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 集合上的最近点对的问题, 其中两点 (x_i, y_i) 和 (x_j, y_j) 之间的距离是通常的欧几里得距离 $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。这个问题出现在许多应用中, 例如确定某航空控制中心管理的特定高度的空间内最近的一对飞机。怎样以一种有效的方式找到这个最近的点对?

解 为了解决这个问题, 可以首先确定每对点的距离, 然后找到这些距离的最小值。但是, 这种方法需要 $O(n^2)$ 次的距离计算和比较, 因为存在 $C(n, 2) = n(n-1)/2$ 个点对。不过存在一个精致的分治算法, 对于 n 个点可以用 $O(n \log n)$ 次的距离计算和比较求解这个最近的点对问题。这里我们描述的算法归功于米凯尔·萨莫斯(见[PrSa85])。

为了简单起见, 假设 $n = 2^k$, 其中 k 是正整数。(我们避免某些当 n 不是 2 的幂时必须考虑的技术)。当 $n = 2$ 时, 只有一对点。在这两个点之间的距离就是最小距离。在算法的开始我们使用归并排序两次, 一次用于依据 x 轴坐标对节点进行升序排序, 一次用于依据 y 轴坐标对节点进行升序排序。每一排序操作需要 $O(n \log n)$ 次运算。我们将在每一次递归步骤中使用这些排序表。

算法的递归部分将问题划分成两个子问题, 每个涉及一半的点。使用按 x 轴坐标对节点进行排序的列表, 画一条垂线将 n 个点分成两部分, 左半部分和右半部分大小相等, 每部分包含 $n/2$ 个点, 如图 1 所示。(如果有任何点落到划分线上, 必要时, 我们把它分在这两部分里。) 在后面的递归步骤我们不再需要根据 x 坐标排序, 因为我们可以从所有的点中选择对应的排序子集。这个选择是可以用 $O(n)$ 次比较完成的任务。



在这个图示中, 在 16 个点的集合中寻找最近点对的问题归约成两个在 8 个点的集合中寻找最近点对的问题和确定中心在 ℓ 宽为 $2d$ 的间隙中是否存在比 $d = \min(d_L, d_R)$ 更近的点的问题

图 1 求解最近点对问题的算法的递归步骤

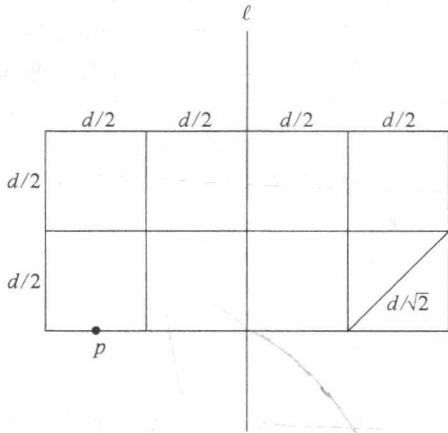
最近的点对的位置有三种可能: 两点都在左部区域 L ; 两点都在右部区域 R ; 一点在左部区域且另一点在右部区域。递归地使用这个算法计算 d_L 和 d_R , 其中 d_L 是在左部区域的点之间

的最小距离, d_R 是在右部区域的点之间的最小距离, 令 $d = \min(d_L, d_R)$ 。为了成功地将原始集合找最近点对的问题划分成在两个区域分别找最短距离的问题, 我们必须处理算法的分割之后的治理部分, 这要求我们考虑最近的点处在不同的区域的情况, 即一点在 L 中, 另一点在 R 中。因为存在一对距离为 d 的点, 所以或者它们都在 R 中, 或者它们都在 L 中。对于分在不同区域的最近的点, 要求其距离一定小于 d 。

如果一点在左边区域, 一点在右边区域且处在小于 d 的距离内, 那么这些点一定位于宽度 $2d$ 的以线 ℓ 作为其中心的垂直带状区域中。(否则, 这些点的距离一定大于它们的 x 坐标之差, 而这个距离将超过 d 。)为了检查在这个带状区域中的点, 我们对它们进行排序并按照 y 坐标递增的顺序把它们列出来。这可以使用归并排序用 $O(n \log n)$ 次运算完成, 并且只需在算法开始时做一次, 而不是在每个递归步做。在每个递归步, 我们从已经按照其 y 坐标排好序的所有点的集合, 构造在这个区域内的根据其 y 坐标排序的点的子集, 这可以用 $O(n)$ 次比较完成。

从带状区域中具有最小 y 坐标的一个点开始, 我们连续地检查带状区域中的每个点, 计算这个点与带状区域中所有其他具有较大 y 坐标且与这个点的距离小于 d 的点之间的距离。注意为检查点 p , 我们只需要考虑在 p 和下述矩形中的一组点之间的距离。这个矩形的高为 d , 宽为 $2d$, p 在它的底边上, 并且它的垂直边与 ℓ 的距离为 d 。

我们可以证明在这个点集中至多存在 8 个点, 其中包含 p 在内(或者在这个 $2d \times d$ 的矩形的边上)。为了看到这一点, 注意在图 2 所示的 8 个 $d/2 \times d/2$ 的正方形中, 每个正方形内部至多可能存在一个点。这是由于在一个正方形的边上或者内部最远距离的点是到对角线的长度 $d/\sqrt{2}$ (使用勾股定理可以得到), 这个距离小于 d , 并且每个这样的正方形是完全处在左区域内或者右区域内。这意味着在这一步我们至多只需要与 d 比较 7 个距离, 这些距离是在 p 和矩形内部或者边上 7 个或者更少的其他的点之间。



包含 p 在内, 至多 8 个点可能处在中心在 ℓ 的 $2d \times d$ 的矩形内或者边上, 因为在 8 个 $(d/2) \times (d/2)$ 的正方形中, 每个内部或边上至多可能存在一个点

图 2 说明对带状区域中的每个点至多需要考虑另外 7 个点

由于在宽为 $2d$ 的带状区域中的总点数不超过 n (集合中的总点数), 所以至多需要与 d 比较 $7n$ 个距离以找到点之间的最小距离。即只存在 $7n$ 个距离可能小于 d 。因此, 一旦用归并排序按照这些点的 x 坐标和 y 坐标对它们进行排序后, 我们发现求解最接近点对问题需要的比较次数不超过满足递推关系

$$f(n) = 2f(n/2) + 7n$$

的增函数 $f(n)$, 其中 $f(2) = 1$ 。根据定理 2, 得到 $f(n)$ 是 $O(n \log n)$ 。用归并排序算法根据点的 x 坐标和 y 坐标对点做两次排序, 每次排序用 $O(n \log n)$ 次比较, 在算法的 $O(\log n)$ 个步中的每一步, 这些坐标的排序子集每次可以用 $O(n)$ 次比较得到。因此, 这个最近点对问题可以用 $O(n \log n)$ 次比较求解。